An empty directory without index which error is in NGNIX?

In Nginx, if a directory does not contain an `index.html` file (or any other file specified as the index file in the Nginx configuration) and directory listing is turned off, accessing that directory via a browser typically results in a **403 Forbidden** error. This happens because the server is configured to prevent the listing of the directory contents and there's no index file to display as a default page.

404 is not better?

The choice between returning a 404 Not Found error and a 403 Forbidden error when a directory is accessed without an index file and directory listing is disabled depends on the intended message and security policy.

- **404 Not Found**: This error implies that the URL points to a resource that does not exist. Using a 404 error might be more user-friendly or less revealing about the server's structure, as it doesn't explicitly indicate that the directory exists. It simply states that the requested resource is unavailable, which could be interpreted as the directory or page not existing at all.
- **403 Forbidden**: This error explicitly tells the user that the server understood the request but refuses to authorize it. If a directory without an index file is accessed and directory listings are disabled, a 403 error is technically correct because the resource (directory listing or index file) is there but access to it is not allowed. This provides a clear indication that the directory exists but the server's configuration prevents its contents from being listed or an index file from being shown.

Choosing between a 404 and a 403 error often depends on the desired security posture and user experience. For example, if you prefer to minimize information about the server’s directory structure and potential points of vulnerability, a 404 might be the better choice. On the other hand, a 403 error might be used when it's okay to reveal the

existence of a directory but not its contents, often used in more secure or private areas of a website.

What is the actual domain of a website www.example.com or example.com?

The actual domain of a website like "example.com" is technically referred to as the "second-level domain," which in this case is "example.com." This is the main, registrable domain under which different subdomains can be hosted.

"www.example.com ↗" is a specific subdomain of "example.com." The "www" part is a commonly used subdomain that historically stood for "World Wide Web" and was used to clearly designate web servers or the web-facing part of a service. However, it's just one of many possible subdomains and not necessary for a website to function—many modern websites now operate without the "www" and simply use the second-level domain directly.

Both "www.example.com ↗" and "example.com" can be used to access a website, but they are technically treated as separate hosts by DNS systems. It's common practice to redirect one to the other to maintain consistency; for example, redirecting "www.example.com ↗" to "example.com" or vice versa, depending on the organization's branding and SEO strategies. This helps to avoid any confusion or duplication in search engine indexing and user access.

But the original one actually is example.com right? But if we store a website in a web server is more correct to store the website in a folder www.example.com or example.com? www.example.com means that this is the folder where we have the html stuff related to the domain and not the ftp or smtp stuff right?

Yes, "example.com" is the root or base domain and is technically the "original" or primary domain. When setting up a website and organizing server directories, whether to use "www.example.com ↗" or just "example.com" for storing website files can depend on several factors:

1. **Server Configuration and Naming Conventions**: On many web servers, especially those hosting multiple sites, it is common to create directories named after the full domain that the website will use, including subdomains like "www". This helps in organizing and isolating the content of different sites and subdomains clearly. For example, if a server hosts "www.example.com ↗" and "blog.example.com", having separate folders for each can help manage their data independently.
2. **Purpose of Subdomains**: If you're using multiple subdomains for different services (like "ftp.example.com" for FTP services, "mail.example.com" for email services, and "www.example.com ↗" for the website), it makes sense to organize each service's files in separate directories named according to the subdomain. This helps keep the structure clear and ensures that configurations do not overlap or interfere with one another.
3. **Redirection and Accessibility**: If "example.com" and "www.example.com ↗" are both used to access the same site, it doesn't matter much in terms of directory naming, as long as proper redirects are configured. For instance, you might store all website files in a folder named "www.example.com ↗" and set up a server redirect from "example.com" to "www.example.com ↗" (or vice versa). This setup maintains a clear distinction in the server files and supports consistent URL usage for SEO and user access.

In practical terms, naming the directory "www.example.com ↗" might indeed suggest that this is specifically for the web content (HTML, CSS, JavaScript, etc.), and it can help differentiate web content management from other types of content like FTP or SMTP configurations, which might be handled elsewhere or through other subdomains.

Ultimately, the decision on how to name directories and organize server files should align with the administrative and operational strategies of the server, as well as the clarity and ease of management for the system administrators.